

Algorytmy Metaheurystyczne

Sprawozdanie z Laboratorium 1

Sara Żyndul

14 kwietnia 2026

1 Wstęp i opis problemu

Problem komiwojażera (ang. Travelling Salesman Problem, TSP) polega na znalezieniu najkrótszego cyklu Hamiltona w ważonym grafie, czyli najkrótszej trasy odwiedzającej każde miasto (wierzchołek) dokładnie jeden raz i wracającej do punktu startowego.

W ramach obecnego ćwiczenia badano algorytmy heurystyczne z rodziny przeszukiwania lokalnego (ang. *Local Search*). Heurystyki te opierają się na założeniu, że optymalne decyzje lokalne prowadzą do dobrych rozwiązań globalnych. Algorytm w każdej iteracji eksploruje zdefiniowane otoczenie (sąsiedztwo) aktualnego rozwiązania i przechodzi do sąsiada, jeśli ten oferuje mniejszy koszt (krótszą trasę), aż do momentu, w którym nie da się już poprawić rozwiązania (utknięcie w minimum lokalnym). Metoda ta jest zazwyczaj szybsza od algorytmów dokładnych.

2 Metodologia

W eksperymentach przetestowano dziesięć zbiorów danych (instancji problemu), od małych (np. dj38, wi29) do bardzo dużych (np. ca4663, ei8246). Przebadano następujące warianty algorytmów:

1. **Standardowy Local Search:** Zaczyna od losowej permutacji. Przeszukuje całe sąsiedztwo i wykonuje krok do najlepszego dostępnego rozwiązania, powtarzając proces aż do znalezienia minimum lokalnego.
2. **Local Search Fast:** Zamiast przeszukiwania całego sąsiedztwa, bada jedynie n losowo wybranych sąsiadów i wybiera najlepszego spośród nich, co w praktyce przyspiesza wykonanie pojedynczego kroku.
3. **MST + Local Search:** Na początku wyznaczane jest Minimalne Drzewo Rozpinające (MST) algorytmem Prima. Następnie przez przejście DFS konstruowany jest cykl początkowy (tzw. 2-aproksymacja), po czym aplikowany jest algorytm Local Search, który iteracyjnie skraca uzyskaną trasę.

3 Wyniki eksperymentalne

Zebrane wyniki zostały opracowane w Tabeli 1 (przedstawiającej ostateczny średni koszt trasy względem trasy optymalnej/optimal-ish) oraz w Tabeli 2 (ukazującej średnią liczbę iteracji/kroków potrzebnych do zakończenia algorytmu).

Tabela 1: Zestawienie średnich kosztów tras dla badanych instancji

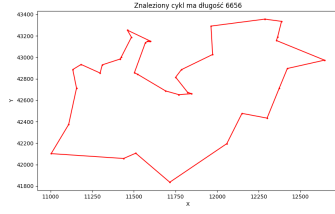
Instancja	Optimal Tour	Koszt MST	Local Search	LS Fast	MST + LS
dj38	6 656	5 828	6 956	9 953	6 719
wi29	27 603	21 753	28 256	44 050	28 900
qa194	9 352	8 026	10 223	16 194	10 126
uy734	79 114	69 873	86 921	168 348	84 051
zi929	95 345	82 863	104 190	150 528	100 435
mu1979	86 891	74 109	94 045	173 016	92 978
ca4663	1 290 319	1 118 421	1 428 915	2 671 409	1 369 259
tz6117	394 718	348 447	436 651	871 057	423 041
eg7146	172 387	147 304	189 901	364 839	184 521
ei8246	206 171	185 353	229 971	524 120	219 760

Tabela 2: Zestawienie średniej liczby kroków poprawy

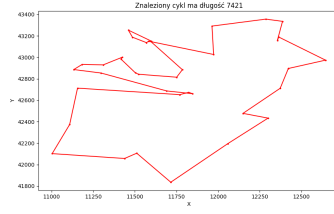
Instancja	Local Search	LS Fast	MST + LS
dj38	34.00	31.29	10.00
wi29	25.00	15.50	7.17
qa194	210.86	197.43	42.79
uy734	924.29	815.54	171.36
zi929	1176.06	1258.87	204.77
mu1979	2793.87	2644.69	417.62
ca4663	6776.38	6659.43	985.26
tz6117	8881.88	8644.57	1311.88
eg7146	10263.50	9806.19	1596.38
ei8246	12008.00	10922.60	1795.38

4 Wizualizacje tras

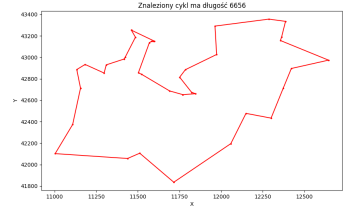
Poniżej zamieszczono wizualizacje działania każdego z trzech wariantów na analizowanych instancjach:



(a) Local Search

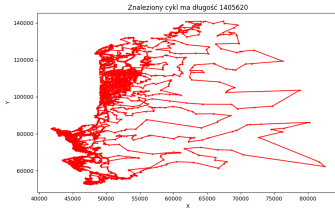


(b) LS Fast

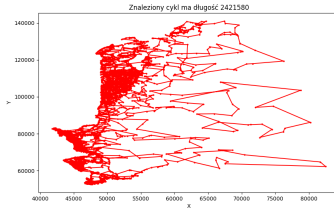


(c) MST + LS

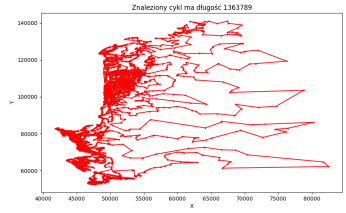
Rysunek 1: Wizualizacje dla instancji dj38



(a) Local Search

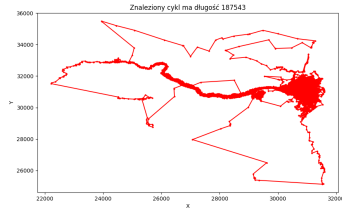


(b) LS Fast

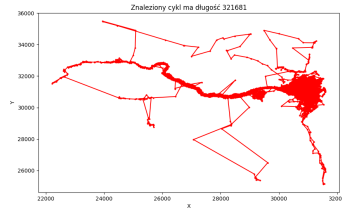


(c) MST + LS

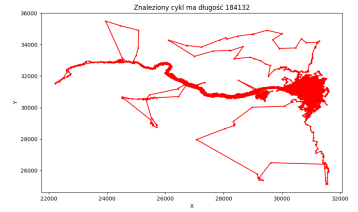
Rysunek 2: Wizualizacje dla instancji ca4663



(a) Local Search

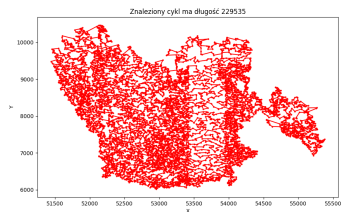


(b) LS Fast

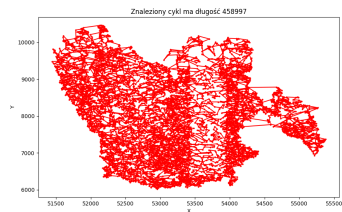


(c) MST + LS

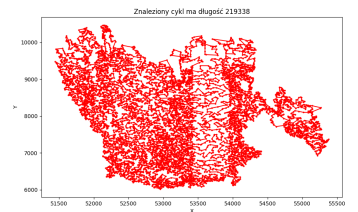
Rysunek 3: Wizualizacje dla instancji eg7146



(a) Local Search

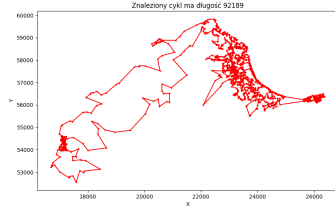


(b) LS Fast

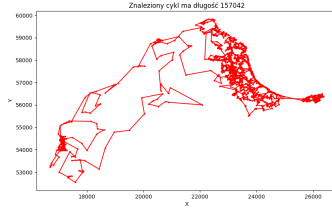


(c) MST + LS

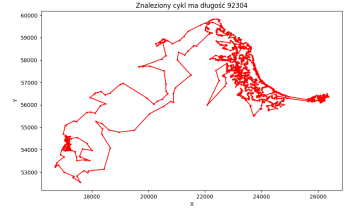
Rysunek 4: Wizualizacje dla instancji ei8246



(a) Local Search

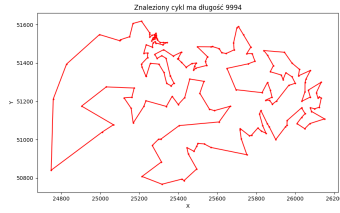


(b) LS Fast

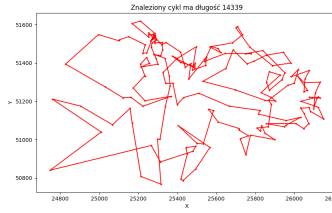


(c) MST + LS

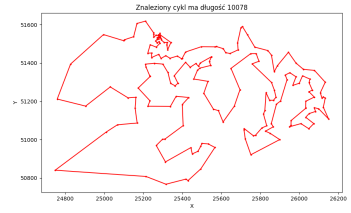
Rysunek 5: Wizualizacje dla instancji mu1979



(a) Local Search

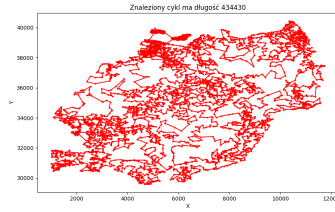


(b) LS Fast

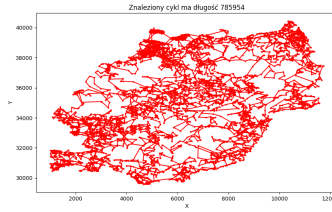


(c) MST + LS

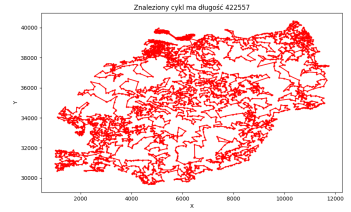
Rysunek 6: Wizualizacje dla instancji qa194



(a) Local Search

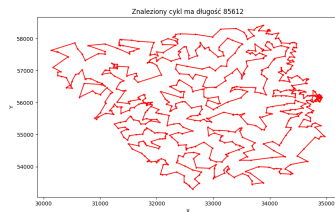


(b) LS Fast

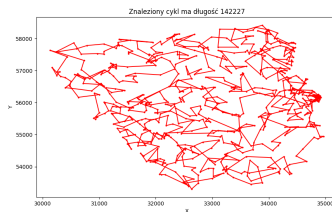


(c) MST + LS

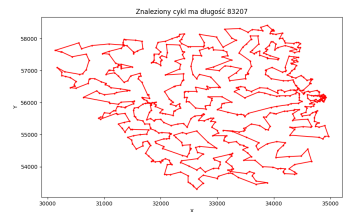
Rysunek 7: Wizualizacje dla instancji tz6117



(a) Local Search

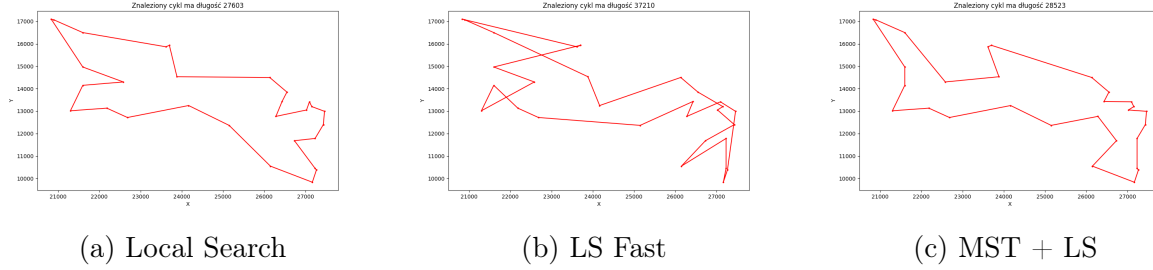


(b) LS Fast

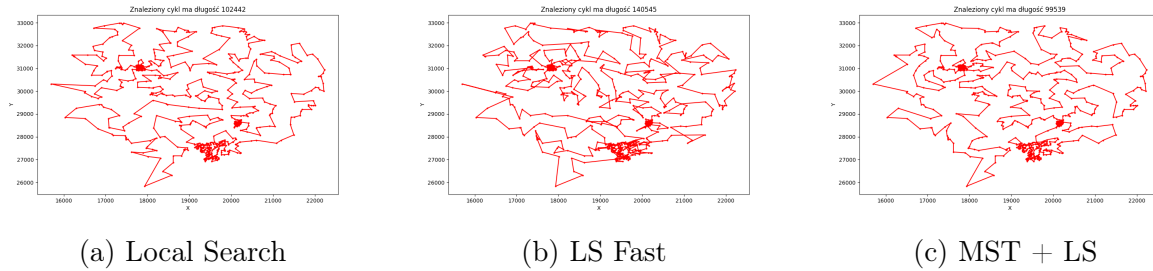


(c) MST + LS

Rysunek 8: Wizualizacje dla instancji uy734



Rysunek 9: Wizualizacje dla instancji wi29



Rysunek 10: Wizualizacje dla instancji zi929

5 Wnioski z analizy wyników

Na podstawie przedstawionych danych można sformułować następujące kluczowe wnioski:

1. **Przewaga rozwiązania początkowego (MST + Local Search):** Algorytm rozpoczynający poprawę od cyklu skonstruowanego na bazie Minimalnego Drzewa Rozpinającego uzyskał bezapelacyjnie najlepsze wyniki dla każdej z badanych instancji. Wartości kosztów uzyskane tą metodą są najbliższe trasom uznanym za optymalne ("Optimal Tour"). Pokazuje to jak istotny jest wybór dobrego punktu startowego przy problemach NP-trudnych.
2. **Niska efektywność wariantu Fast:** Algorytm wariantu Local Search Fast, mający za zadanie przyspieszyć proces obliczeniowy na poziomie jednego kroku (sprawdzanie tylko n sąsiadów), znacząco pogarsza końcowy wynik we wszystkich testach. Koszty wygenerowanych tras są w nim często blisko dwukrotnie wyższe od tras optymalnych, a algorytm szybko zbiega i "utknął" w słabym minimum lokalnym.
3. **Liczba iteracji poprawy:** Najszybciej konvergującym wariantem (wymagającym najmniejszej liczby kroków zmiany rozwiązania) jest MST + Local Search. Wynika to z faktu, że algorytm startuje już ze stosunkowo wysokiej jakości cyklu, przez co do lokalnego optimum wystarczy mu relatywnie mało ulepszeń. Dla instancji *ca4663* standardowy Local Search wymagał blisko 6776 iteracji, podczas gdy po zbudowaniu MST potrzebne było tylko 985, uzyskując jednocześnie krótszą i znacznie lepszą całkowitą długość trasy.